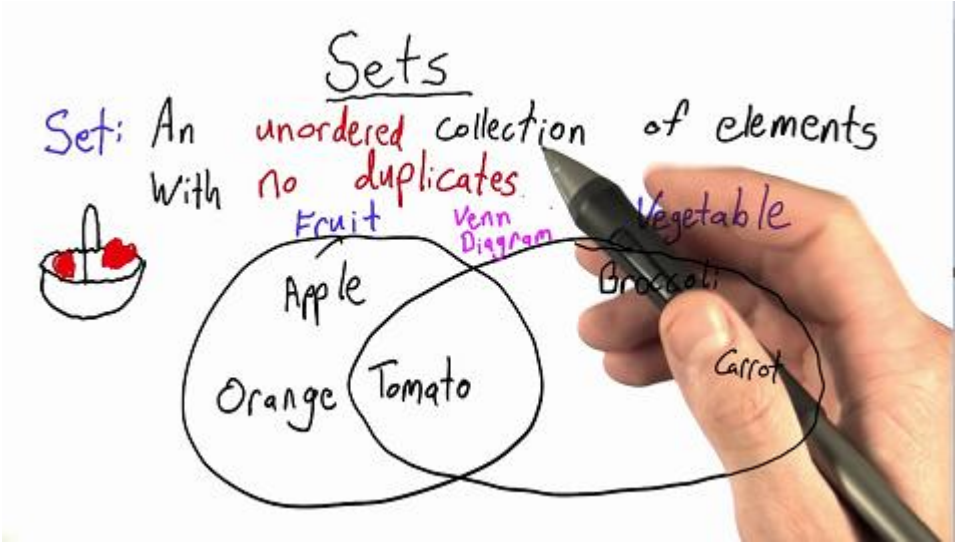
- Colecciones de Python

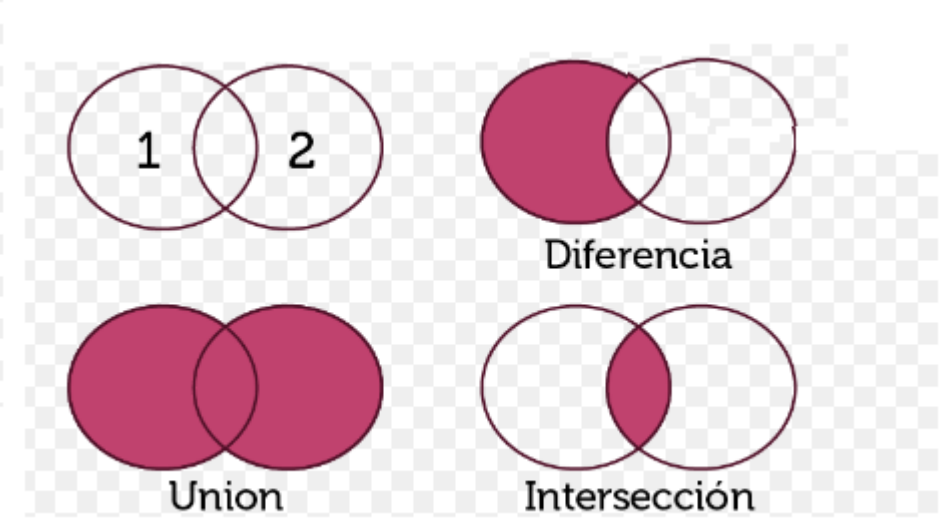
  Conjuntos

  Análisis de Textos

  Expresiones Regulares

6.3. Conjuntos





Como funcionan los conjuntos ?

set ()

.add

Colección de elementos únicos.

```
In [1]: conjunto = set()

In [2]: conjunto
Out[2]: set()

In [3]: conjunto = {1,2,3}

In [4]: conjunto
Out[4]: {1, 2, 3}

In [5]: conjunto.add(4)

In [6]: conjunto
Out[6]: {1, 2, 3, 4}
```

```
In [7]: conjunto.add(0)

In [8]: conjunto
Out[8]: {0, 1, 2, 3, 4}

In [9]: conjunto.add('H')

In [10]: conjunto
Out[10]: {0, 1, 2, 3, 4, 'H'}

In [11]: conjunto.add('A')

In [12]: conjunto.add('Z')

In [13]: conjunto
Out[13]: {0, 1, 2, 3, 4, 'A', 'Z', 'H'}
```

```
In [14]: grupo = {'Hector','Juan','Mario'}

In [15]: 'Hector' in grupo
Out[15]: True

In [16]: 'Maria' in grupo
Out[16]: False

In [17]: 'Hector' not in grupo
Out[17]: False

In [18]: test = {'Hector','Hector','Hector'}

In [19]: test
Out[19]: {'Hector'}
```

in = busca en el conjunto

list = pasa conjunto a lista

set (lista) = pasa lista a conjunto

```
In [20]: l = [1,2,3,3,2,1]
         l
Out[20]: [1, 2, 3, 3, 2, 1]

In [21]: c = set(l)

In [22]: c
Out[22]: {1, 2, 3}

In [23]: l = list(c)

In [24]: l
Out[24]: [1, 2, 3]
```

```
In [25]: l = [1,2,3,3,2,1]

In [26]: l = list( set( l ) )

In [27]: l
Out[27]: [1, 2, 3]
```

Truco para eliminar duplicados de una lista :

1. Convierte la lista a conjunto

2. Convierte el conjunto a lista

Truco:

Convertir una cadena en un conjunto para eliminar letras duplicadas

```
In [28]: s = "Al pan pan y al vino vino"
         set(s)
Out[28]: {' ', 'A', 'a', 'i', 'l', 'n', 'o', 'p', 'v', 'y'}
```

Ejemplo – Conjunto de usuarios

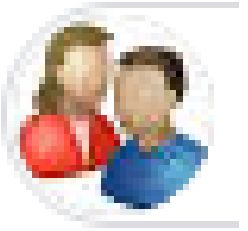
Realiza un programa que siga las siguientes instrucciones:

1. Crea un conjunto llamado usuarios con los usuarios Marta, David, Elvira, Juan y Marcos
2. Crea un conjunto llamado administradores con Juan y Marta.
3. Borra al administrador Juan del conjunto de administradores.

```
#1.
usuarios = { 'Marta', 'David', 'Elvira', 'Juan' , 'Marcos' }
type (usuarios)

#2.
administradores  = { 'Marta',  'Juan'  }

#3.
administradores -= {'Juan'}
print (administradores)
```



4. Añade a Marcos como un nuevo administrador, pero no lo borres del conjunto de usuarios.
5. Muestra todos los usuarios e indica si cada usuario es administrador o no.

2. **Avanzado.** Crea un Menú con 4 opciones

1. Crear Usuario,
2. Borrar usuario,
3. Añadir al grupo Administradores
4. Quitar del grupo de Administradores



- Si la opción es 1 pide el nombre del usuario y añádelo a la lista
- Si la opción es 2 elimina el usuario
- Si la opción es 3 o 4 muestra la lista de usuarios y la de administradores, pide un usuario existente para añadir o quitar del grupo de administradores.

Métodos de los conjuntos

Métodos básicos

- add() Añade un elemento a un conjunto
- discard() elimina un elemento si lo encuentra
- copy() copia un conjunto en otro
- clear() vacía un conjunto

Comparación de conjuntos

- isdisjoint() Son disjuntos ?
- issubset() Es un subconjunto ?
- issuperset() Es un supra-conjunto ?

Métodos avanzados

- union() Unión de 2 conjuntos
- update() Añade elementos
- difference() Diferencia de 2 conjuntos
- difference_update()
- intersection() Intersección de 2 conjuntos
- intersection_update()
- symmetric_difference() Estan en uno o en otro pero no en los 2

https://en.wikibooks.org/wiki/Python_Programming/Sets

Copia a tu cuaderno

```
c1 = set ()                          # Un nuevo conjunto vacío
c1.add ("gato")                      # Agregar un solo elemento
c1.update (["perro", "ratón"])       # Agregar varios elementos,
                                     # por extensión de la lista
c1 |= set (["ciervo", "caballo"])    # Agregar 2 elementos
if "gato" in c1 :                    # Esta en la colección ?
  c1.remove ("gato")
# c1.remove ("elefante") arroja un error
c1.discard ("elefante")              # No lanza ningún error
print (c1)

for item in c1:                      # Iteracion for each element
  print (item)
print ("Item count:", len(c1))       # Longitud /tamaño/núm.elementos
                                     # Error: no  hay índices para conjunt
es_vacio = len(c1) == 0              # Prueba de vacío

c1 = {"gato", "perro"}               # Inicializar el set usando llaves;
#c1 = {}                             # Esto no es un diccionario
c1 = set(["gato", "perro"])          # Inicializar  usando una lista
c2 = set(["perro", "raton"])
c3 = c1 & c2                         # Intersección
```

```
c4 = c1 | c2                # Union
c5 = c1 - c3                # Diferéncia de conjuntos
c6 = c1 ^ c2                # Diferéncia simétrica (opuesto a
                            #    intersección
issubc = c1 <= c2           # Prueba de subconjunto
issuperc = c1 >= c2         # Prueba de superconjunto

c7 = c1.copy()              # Crea una copia
c7.remove("gato")
print (c7.pop())            # Elimina un elemento arbitrario

c8 = c1.copy()
c8.clear()                  # Limpia, vacia el conjunto

#----------------- Conjunto generado por compresión con un for
c9 = {x for x in range(10) if x % 2}

print (c1, c2, c3, c4, c5, c6,
c7, c8, c9, issubc, issuperc)
```

```
c9a =set()
for x in range(10):
    if x % 2 :
        c9a.add(x)
print(c9a)
```

Practica P01 – Métodos de los conjuntos

Ejercicio 1

Sea el conjunto c1 = {1, 2, 3, 4}

a. Añade el 5, descarta el 2 usando métodos. (add, discard)

b. Agrega 3 números aleatorios (random) del 1 al 20

(de random : `sample, randrange`)

c. Crea otro conjunto llamado c2 con los cuadrados del conjunto c1.

d. Calcula e imprime la unión, intersección y diferencia de los conjuntos c1 y c2.

(*) import random .

random.randrange(1, 20)

Ejercicio 2

Crea una función que se llame “**modificar**” que partir de una lista de números realice las siguientes tareas sin modificar el original

1. Borrar los elementos duplicados : pasando a conjunto y luego a lista otra vez
2. Ordenar la lista de mayor a menor : ordenar la lista con sort
3. Eliminar todos los números impares : creando una nueva lista con los que cumplen
4. Realizar una suma de todos los números que quedan : función sum
5. Añadir como primer elemento de la lista la suma realizada : insert o slicing
6. Devolver la lista modificada return

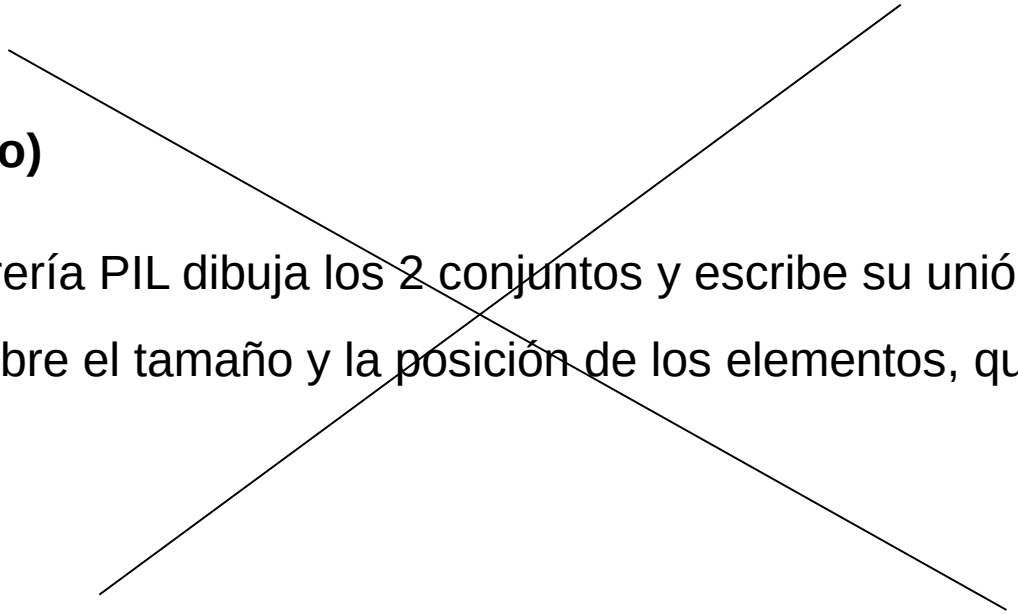
Finalmente, después de ejecutar la función, comprueba que la suma de todos los números a partir del segundo concuerda con el primer número de la lista. Ejemplo: [1,2,3] -> [6,1,2,3]

Ejercicio 3 -

Pide por pantalla dos cadenas de elementos separadas por comas.

- Pásalo a listas (lista1, lista2) haciendo Split de cada cadena.

• Pásalo a conjuntos y calcula e imprime : su unión e intersección.



(Muy Avanzado)

• Usando la librería PIL dibuja los 2 conjuntos y escribe su unión e intersección

• Reflexiona sobre el tamaño y la posición de los elementos, que criterio de cálculo usarías.

Sintaxis de PIL

```
from PIL import Image, ImageDraw
img = Image.new('RGB', (500, 300), (128, 128, 128))
dib = ImageDraw.Draw(img)
dib.ellipse((100, 100, 150, 200), fill=(255, 0, 0),
            outline=(0, 0, 0))

dib.text(xytuple, text, fill=None, font=None, anchor=None,
         spacing=0, align="left")
```

https://note.nkmk.me/en/python-pillow-imagedraw/

```
# SOLUCION P01. Ejercicio 1

from random import sample, randrange
c1 = {1, 2, 3, 4}
c1.add(5)
c1.discard (2)
print(c1)
# https://pynative.com/python-random-randrange/
c1.update ( {x for x in sample(randrange(1, 21), 3)} )
print("c1: ", c1)

c2 = { x**2 for x in c1 }
print("c2: ",c2)

print("c1 | c2: ", c1.union(c2))
print("c1 & c2: ", c1.intersection(c2))
print("c1 - c2: ", c1.difference(c2))
```

```
# SOLUCION P01. Ejercicio 2
#-------------------------------------------------
# Funcion modificar que apartir de una lista de números

def modificar (lista) :

    lis1 = list(set(lista))
    print(">>> sin duplicados :", lis1)
    lis1.sort()
    print (">>> ordenada :", lis1)

    lis2 = [ x for x in lis1 if x % 2 == 0]
    print(">>> solo pares :", lis2)

    lis2.insert(0, sum(lis2))
    print(">>> añadir suma  :", lis2)
    return (lis2)

#------------------------------------ Inicio del programa
lini = [1,2,3,4,5,6,7,8,3,4,8]
lfin = modificar (lini)

print  (lfin[0] == sum(lfin [1::]))
```

SOLUCION P01. Ejercicio 3

```
# str1 = input ("Entri llista 1 (separada per comes) : ")
# lista1 = str1.replace(" ", "").split(",")

# str2 = input ("Entri llista 2 (separada per comes) : ")
# lista2 = str2.replace(" ", "").split(",")

lista1 = ['banana', 'pera', 'manzana']
lista2 = ['baloo', 'banana', 'mowgli']

c1 = set(lista1)
c2 = set(lista2)

print ("c1 : ", c1)
print ("c2 : ", c2)
print ("Union  c1 | c2: ", c1 | c2)
print ("Inter. c1 & c2  : ", c1 & c2)
```

Continua en otra celda, ver diapositiva siguiente …

```
from PIL import Image, ImageDraw
```

segueix
SOLUCION P01. Ejercicio 3

```
img = Image.new('RGB', (600, 400), 'White')
dib = ImageDraw.Draw(img)

tam1 = len(c1)
tam2 = len(c2)

#------------- dibuja una elipse por cada conjunto
font = ImageFont.truetype('arial',14)

dib.ellipse((10, 10, tam1 * 60, tam1 * 60), 'Crimson')
# (x0, y0, x1, y1)
for i, item in enumerate(c1) :
    dib.text((40 + (i * 20) ,40 + (i * 20)),
        str(item), (255,255,255), font=font)

xi = max(300, tam2 * 30)
yi = max(200, tam2 * 30)
dib.ellipse((xi, yi, 550, 350), 'Green')  # (x0, y0, x1, y1)
for i, item in enumerate(c2) :
    dib.text((xi +40 + (i * 20) , yi + 40 +(i * 20)),
        str(item), 'White', font=font)

dib.text((400, 10), "Inters: " +str(c1&c2), 'Red', font=font)
dib.text((400, 70), "Unio: " +str(c1|c2), 'Black', font=font)

img.show()
```

El Análisis de texto automático

- Descartar palabras (limpieza previa)
- Buscar las palabras más usadas (relevancia)
- Detectar conjuntos de palabras (patrones)



Práctica P02-Conjuntos

Realiza los siguientes pasos :

1- Dada la cadena = “`T7 Prom 5'-d CGC CAG GGT TTT TCA AGT CAC GAC GAC AGT TAT TGC TCA GCG G`“

2- Pásalo a formato lista usando el espacio como separador

palabras = cadena.split (“ ”)

3- Muestra las distintas palabras del texto (pasa a conjuntos) (set)

distintas =

4- Ahora recorre distintas con un for, y responde (imprime) :

4.1- Cuántas veces aparece cada palabra

4.2- Cuántas palabras hay de 3 letras

Práctica P03-Conjuntos (avanzado)

Realiza los siguientes pasos :

1- Crea una variable de texto multi-linea, y copia-pega el texto de la diapositiva siguiente

2- Pasa a minúsculas, quita acentos, puntos y comas.

3- Pásalo a formato lista usando el espacio como separador (split)

4- Muestra las distintas palabras del texto (pasa a conjuntos)

5- Elimina las palabras = “el, la, los, las, un, de”

Sigue ...

6- Ahora recorre la lista de palabras distintas con un for, y para las palabras de más de 5 letras :

6.1- Cuántas veces aparece cada palabra

6.2- Cuántas palabras hay de más de 5 letras

```
cadena = """La experiencia de aprendizaje de Python difiere bastante en función de la experiencia de cada uno.
Sea cual sea el lenguaje informático practicado, es necesario tener cierta lógica y ser capaz de dominar ciertos conceptos algorítmicos.
Escoger Python como primer lenguaje es la mejor elección que puede realizar: muy próximo al lenguaje natural y a los conceptos algorítmicos clásicos, le permitirá hacer gran cantidad de cosas de manera muy natural y aprovechar una curva de aprendizaje muy pronunciada.
Esta experiencia de aprendizaje difiere bastante según los lenguajes practicados en el pasado. En efecto, cada lenguaje aporta su propia manera de pensar y su implementación de las técnicas algorítmicas, lo que moldea el pensamiento del que lo practica."""
```

Solución P02 Conjuntos

#P02– Conjuntos----------

```
cadena = 'T7 Prom 5-d CGC CAG GGT TTT TCA AGT CAC GAC GAC AGT TAT
TGC TCA GCG G '

palabras = cadena.split(" ")       # divide cadena en palabras
c = set(palabras)                  # crea conjunto de palabras distintas
print (c)
distintas = list(c)                # crea lista a partir del conjunto

pal3 = 0                  # crea contador para palabra de 3 letras

for  palabra in distintas :     # cuantas veces  se encuentra
   print ( f" {palabra} : {palabras.count(palabra)}  veces " )
   if len(palabra) == 3 :
       pal3 +=1
print (f"Hay {pal3} palabras de 3 letras ")
```

Solución P03

```
#P03----------------------------------------------------------------------------
cadena = "La experiencia de aprendizaje de Python difiere bastante en función de la experiencia de cada uno. \
Sea cual sea el lenguaje informático practicado, es necesario tener cierta lógica y ser capaz de dominar ciertos \
conceptos algorítmicos. \
Escoger Python como primer lenguaje es la mejor elección que puede realizar: muy próximo al lenguaje natural y a \
los conceptos algorítmicos clásicos, le permitirá hacer gran cantidad de cosas de manera muy natural y aprovechar \
una curva de aprendizaje muy pronunciada. \
Esta experiencia de aprendizaje difiere bastante según los lenguajes practicados en el pasado. En efecto, cada \
lenguaje aporta su propia manera de pensar y su implementación de las técnicas algorítmicas, lo que moldea el \
pensamiento del que lo practica."

cadena = cadena.lower().replace(".", "").replace(",", "")
palabras = cadena.split(" ")      # Convierte el texto en una lista
print ("hay : ",len(palabras))    # Cuantas plabras hay
c = set(palabras)                 # convierte lista en conjunto
#conjunto de palabras a eliminar
cexcluye = {"el", "la", "los", "las", "un", "de"} c = c - cexcluye
# y pasa el conjunto a otra lista palabras pero ya distintas
distintas  = list(c)
print ("hay distintas:",len(distintas))
distintas.sort()                  # Ordena la lista
```

```
print ("{:15} {:5} ".format("palabras", "veces"))
print ("{:15} {:5} ".format("--------", "-----"))

palab_mas_5 = []   # Lista de palabras de más de 5 letras

for palabra in distintas :          # para cada palabra diferente

    if len(palabra) > 5 :
        palab_mas_5.append( ( palabras.count(palabra), palabra )  )
        # cuenta cuantas veces aparecia en la lista original

        print ("{:15} {:5}".format(palabra, palabras.count(palabra)))

print ("palabras de más de 5 letras  : ", len(palab_mas_5))

# ------- El print siguiente sale ordenado por frecuencia
palab_mas_5.sort(reverse=True)
print(palab_mas_5)
```

Expresiones Regulares

Una expresión regular es un cadena de caracteres especial que define una búsqueda de patrones.

Puedes pensar en las expresiones regulares como un tipo de comodín con asteriscos.

En Python tenemos la librería re que permite hacer operaciones de búsqueda, y reemplazo sobre expresiones regulares.

Function	Description
findall	Returns a list containing all matches
search	Returns a Match object if there is a match anywhere in the string
split	Returns a list where the string has been split at each match
sub	Replaces one or many matches with a string

```python
import re

text = 'My name is Abder, good name'
res = re.findall('name', text)
print (res)
```

```
['name', 'name']
```

```python
import re
cadena1 = 'casa'
cadena2 = 'casas'
cadena3 = 'pasa'

if re.match(cadena1, cadena2):
    print('cadena1 y cadena2 son coincidentes')
else:
    print('cadena1 y cadena2 no son coincidentes')

if re.match(cadena1, cadena3):
    print('cadena1 y cadena3 son coincidentes')
else:
    print('cadena1 y cadena3 no son coincidentes')
```

```
cadena1 y cadena2 son coincidentes
cadena1 y cadena3 no son coincidentes
```

Info extra : palabras que …

Puedes pensar en las expresiones regulares como un tipo de comodín con asteriscos.

/hola/ : Buscar el texto explícito hola.

/a[nr]t/ : Busca un patrón el cual su primera letra sea la a y su última letra sea la t, y entre dichas letras contenga la letra n o la r.
Por lo tanto, casos de éxito para este patrón, serían las palabras inglesas ant y art.

/ca[tbr]/ : Busca palabras que empiecen con ca y terminen con alguno de estos caracteres tbr

El módulo re de Python incluye funciones para trabajar con expresiones regulares.

re.findall: busca todas las coincidencias en una cadena

La `findall()` función devuelve una lista que contiene todas las coincidencias.

Imprime una lista de todos los partidos:

Ejemplo

```
import re

str = "The rain in Spain"
x = re.findall("ai", str)
print(x)
```

```
import re
text = 'My name is Abder, good name'
res = re.findall(r"[A-Z]", text)
print (res)
```

```
['M', 'A']
```

Patrón : de la A a la Z mayúsculas

```
import re
text = 'My name is Abder, good name'
res = re.findall(r"[A-Z][a-z]", text)
print (res)
```

```
['My', 'Ab']
```

Patrón : de la A a la Z mayúsculas, seguido de una minúscula

```
import re
text = 'My name is Abder, good name'
res = re.findall(r"[A-Z][a-z]{1,4}", text)
print (res)
```

```
['My', 'Abder']
```

Patrón : de la A a la Z mayúsculas, seguido de una a 4 minúscules

Ojo con el rango : es una coma !!!!

```python
import re

str = "The rain in Spain"
res = re.findall(r"[a-z][a][i][a-z]{1,6}", str)
print (res)
```

```
['rain', 'pain']
```

Ejercicio : Cambia el patrón para que te devuelva palabras que contengan ai, en cualquier posición, y que empiecen por minúscula o mayúscula.

Practica conjunta

- Crea y abre el fichero un fichero de texto en modo lectura.
- Pasa todo el texto a minúsculas.
- Busca y muestra todas las palabras de más de 4 letras, y muestra su frecuencia, usando un diccionario para guardar las veces que aparece cada palabra.
- Observa el resultado y valora si las palabras más frecuentes te servirían para etiquetar conceptualmente el texto.

```
import re

#-------------------------------------------------------------------
# Localiza en un texto la frecuencia de las palabras de más de 6 letras
#-------------------------------------------------------------------
veces = {}                          # crea un diccionario para veces de cada
palabra
fiche = open('./dat/texto2.txt', 'r', encoding='utf-8')
texto = fiche.read().lower()
print("texto base", texto)

# obtiene una lista de las palabras de 6 a 15 letras
lista_palabras = re.findall(r'[a-z]{6,15}', texto, re.MULTILINE)

print ("Total palabras de más de 6 letras: ", len(lista_palabras))
# crea un diccionario con cada palabra y su contador de veces
for p in lista_palabras:
    count = veces.get(p,0)
    veces[p] = count + 1

claves = veces.keys()

for p in claves:
    print ( p, ": ", veces[p])
```

Usando re.findall para el buscar en todo el texto multilínea

El módulo re.findall () se usa cuando se desea recorrer en iteración las líneas del archivo, devolverá una lista de todas las coincidencias en un solo paso

re.match: busca un patrón al principio de otra cadena

Split at each white-space character:

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

Search the string to see if it starts with "The" and ends with "Spain":

```
import re

txt = "The rain in Spain"
x = re.search("^The.*Spain$", txt)
```

re.search: busca un patrón en otra cadena:

Replace every white-space character with the number 9:

```
import re

txt = "The rain in Spain"
x = re.sub("\s", "9", txt)
print(x)
```

https://www.w3schools.com/python/python_regex.asp#findall

Identifiers	Modifiers	White space characters	Escape required
\d= any number (a digit)	\d represents a digit.Ex: \d{1,5} it will declare digit between 1,5 like 424,444,545 etc.	\n = new line	. + * ? [] $ ^ () {} | \
\s = space (tab,space,newline etc.)	? = matches 0 or 1	\t =tab	
\S= anything but a space	* = 0 or more	\e = escape	
\w = letters (Match alphanumeric character, including "_")	$ match end of a string	\r = carriage return	
\W =anything but letters (Matches a non-alphanumeric character excluding "_")	^ match start of a string	\f= form feed	
. = anything but letters (periods)	| matches either or x/y	---------- ---	
\b = any character except for new line	[] = range or "variance"		
\.	{x} = this amount of preceding code		

Practica P04

1. Practicando expresiones regulares

Realiza algunos de los ejemplos de la web

https://python-para-impacientes.blogspot.com/2014/02/expresiones-regulares.html

2. Practicando (diccionarios y conjuntos)

Lee este post sobre contar frecuencias de palabras

https://programminghistorian.org/es/lecciones/contar-frecuencias

Practica P05-Nube de Etiquetas

Lee este artículo para aprender a usar la librería WordCloud que genera nubes de etiquetas.
Recuerda limpiar el texto de palabras irrelevantes.

https://juncotic.com/nube-de-palabras-en-python/



Practica P06-Nube II

.Extrae de forma automatizada los textos de diferentes páginas web y crea nubes de palabras personalizadas con Python.

En esta ocasión vamos a ver **cómo podemos crear una nube de palabras** gracias a la librería WordCloud de Andreas Muller (ver web), como por ejemplo esta:

https://antonio-fernandez-troyano.medium.com/nube-de-palabras-word-cloud-con-python-a-partir-de-varias-webs-111e94220822

1. Extraer textos de varias páginas web (web scraping)
2. “Limpiar” los textos (NLTK procesamiento del lenguaje) para eliminar signos de puntuación y palabras demasiado comunes (artículos, pronombres, adverbios, etc)
3. Abrir y convertir una imagen
4. Generar las nubes de palabras con WordCloud

